Tutorial-10
Priority Queue & Hash Table

Evaluative Question

Q1. Consider inserting the keys 10,22,31,4,15,28,17, 88, 59 into a hash table of length m = 11 using open addressing. Illustrate the result of inserting these keys using :

A. **linear probing with h1(k, i) = (k+ i) mod m , where k = the element to be inserted and i is the number of times the collision has occurred. I.e. i starts from 0 and goes until m-1.**
B. **Quadratic probing with h(k,i) = (k +i +3*(i^2)) mod m**
C. **Double hashing with h1(k) =(k mod m) and h2(k) = 1+(k mod(m-1)). Therefore , h(k,i) = (k +i(1+k mod (m-1)))**

Show all intermediate steps and values stored in the hash table

Explanation :

- **Linear probing**:

$h(k,i) = (k+i) \mod 11$	T_0	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
$0 \mod 11$		22	22	22	22	22	22	22	22
$1 \mod 11$								88	88
$2 \mod 11$									
$3 \mod 11$									
$4 \mod 11$				4	4	4	4	4	4
$5 \mod 11$					15	15	15	15	15
$6 \mod 11$						28	28	28	28
$7 \mod 11$							17	17	17
$8 \mod 11$									59
$9 \mod 11$			31	31	31	31	31	31	31
$10 \mod 11$	10	10	10	10	10	10	10	10	10

- **Quadradic probing**, it will look identical until there is a collision on inserting the fifth element:

$h(k,i) = (k + i + 3i^2) \mod 11$	T_0	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
$0 \mod 11$		22	22	22	22	22	22	22	22
$1 \mod 11$									
$2 \mod 11$								88	88
$3 \mod 11$							17	17	17
$4 \mod 11$				4	4	4	4	4	4
$5 \mod 11$									
$6 \mod 11$						28	28	28	28
$7 \mod 11$									59
$8 \mod 11$					15	15	15	15	15
$9 \mod 11$			31	31	31	31	31	31	31
$10 \mod 11$	10	10	10	10	10	10	10	10	10

- **Double hashing**:

$h(k,i) = (k + i(1 + k \mod 10)) \mod 11$	T_0	T_1	T_2	T_3	T_4	T_5	T_6	T_7
$0 \mod 11$		22	22	22	22	22	22	22
$1 \mod 11$								
$2 \mod 11$								
$3 \mod 11$							17	17
$4 \mod 11$				4	4	4	4	4
$5 \mod 11$					15	15	15	15
$6 \mod 11$						28	28	28
$7 \mod 11$								88
$8 \mod 11$								
$9 \mod 11$			31	31	31	31	31	31
$10 \mod 11$	10	10	10	10	10	10	10	10

Practice Questions:

Q1.Consider a hash table with 9 slots and the hash function h(k) = k mod 9. Demonstrate what happens upon inserting the keys 5, 28, 19, 15, 20, 33, 12 , 17 ,10 with collisions resolved by chaining.

Q2.A Priority-Queue is implemented as a Max-Heap. The level-order traversal is 10, 8, 5, 3, and 2. Elements 1 and 7 are inserted in a heap in, respectively. The level-order traversal of the heap will be:-

A)10, 8, 7, 5, 3, 2, 1
B)10, 8, 7, 2, 3, 1, 5
C)10, 8, 7, 1, 2, 3, 5
D)10, 8, 7, 3, 2, 1, 5

Q3. Q1. You use a hash function h to hash n distinct keys into an array T of length m. Assuming independent uniform hashing, what is the expected number of collisions?

Explanaion:

Under the assumption of simple uniform hashing, we will use linearity of expectation to compute this. Suppose that all the keys are totally ordered $\{k_1, \ldots, k_n\}$. Let X_i be the number of $\ell > k_i$ so that $h(\ell) = h(k_i)$. Note, that this is the same thing as $\sum_{j>i} Pr(h(k_j) = h(k_i)) = \sum_{j>i} 1/m = (n-i)/m$. Then, by linearity of expectation, the number of collisions is the sum of the number of collisions for each possible smallest element in the collision. The expected number of collisions is $\sum_{i=1}^{n} \frac{n-i}{m} = \frac{n^2 - \frac{n(n+1)}{2}}{m} = \frac{n^2-n}{2m}$

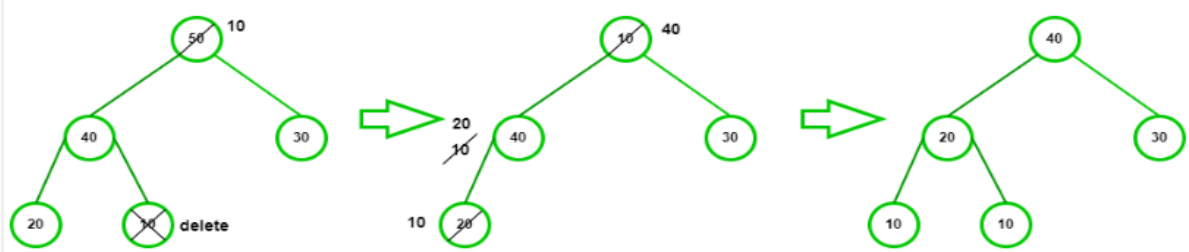
Q. Let A be a priority queue for maintaining a set of elements. Suppose A is implemented using a max-heap data structure. The operation EXTRACT-MAX(A) extracts and deletes the maximum element from A. The operation INSERT(A, key) inserts a new element key in A. The properties of a max-heap are preserved at the end of each of these operations. When A contains n elements, which one of the following statements about the worst-case running time of these two operations is TRUE?

(A)Both EXTRACT-MAX(A) and INSERT(A, key) run in O(1).
(B)Both EXTRACT-MAX(A) and INSERT(A, key) run in O(log(n)).
(C)EXTRACT-MAX(A) runs in O(1), whereas INSERT(A, key) runs in O(n).
(D)EXTRACT-MAX(A) runs in O(1), whereas INSERT(A, key) runs in O(log(n)).

Explanation : for answer B

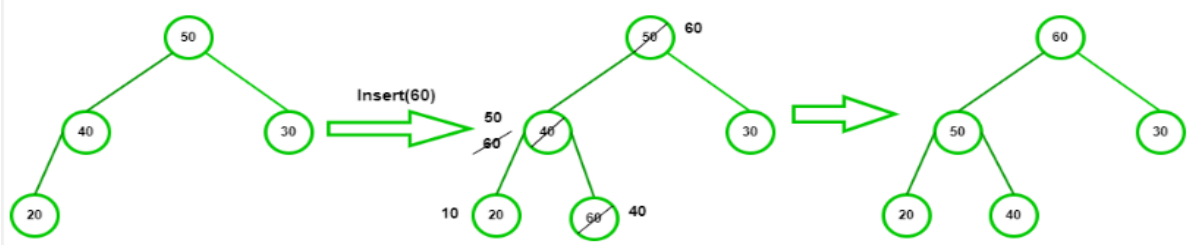
Answer: (B)

Explanation:
Extract_Max(A): Extracts and deletes the maximum element from A



img1

Delete the root node data and replaced it by last level leaf data to maintain the max-heap property, perform top to bottom max-heapify operation, it will take O(logn) time. Insert(A, Key): Inserts a new element key in A.



img2

Insert an element to the last level of max heap. If the inserted value is larger than parent node, then we have to perform bottom-to-top max heapify and it will take O(logn) time. So, both extract max(A) and insert(A, key) run in O(logn) time.